Introduction

Thank you for your interest in the AI Engineer role at Nexthink! We're excited you've chosen to move forward in our selection process. We believe the best way to understand how candidates approach real-world challenges is through practical exercises. This take-home assessment is designed to give you an opportunity to showcase your creativity, depth of knowledge, and ability to think outside the box when solving problems similar to those you'll face at Nexthink. There is no single "right" answer—our primary goal is to understand your reasoning, innovative thinking, and the way you approach complex scenarios. We encourage you to leverage your expertise and creativity to demonstrate how you address requirements, design solutions, and articulate your thought process. Your submission will be evaluated on its technical correctness, design quality, creativity, and the effectiveness of your problem-solving approach. Good luck, and most importantly, have fun with the challenges! We look forward to seeing your innovative ideas in action.

Project Description:

Your task is to develop a real-time newsfeed system that aggregates IT-related news items from selected public sources, filters them for relevance, and provides an easy-to-use interface to display the latest updates. This solution will help IT professionals quickly identify important news such as cybersecurity incidents, software issues, or major disruptions.

Requirements:

Data Aggregation & Ingestion:

- Continuously fetch IT-related news from at least:
  - One IT-focused Reddit subreddit.
  - One or two IT news websites (e.g., Tom’s Hardware, Ars Technica,...).
- To keep evaluation self-contained, our test harness will also inject a synthetic batch through the provided Mock Newsfeed API
- Design your system modularly, allowing new sources to be added or removed easily.

Content Filtering:

- Implement a mechanism to identify news items specifically relevant to IT managers (e.g., outages, security issues, major bugs).
- Propose and justify your chosen filtering approach clearly in your reflection.

User Interface:

- Provide a simple user interface (terminal-based or simple web dashboard).
- Clearly display filtered news, ranked by relevance and recency.

Automated Evaluation Interface (Mock Newsfeed API)

To let our test harness inject synthetic events and verify your filtering logic, expose one minimal, language-agnostic API that supports (a) ingestion of raw items and (b) retrieval of the items your filter accepted. You may implement the interface as a REST endpoint, or any preferred protocol but it must follow the contract below so that we can drive it from our automated tests.

API contract:

Endpoint	Use	Description
/ingest	**Ingest raw events**	Accept **a single call** delivering a JSON array (or stream) of event objects.Each object **must** contain the keys:• `id` (string, unique)• `source` (string, e.g. “reddit” or “ars-technica”)• `title` (string)• `body` (string, optional)• `published_at` (ISO-8601/RFC 3339 timestamp, UTC)The call returns an acknowledgment (HTTP 200 / successful exit status / ACK message).
/retrieve	**Retrieve filtered events**	Provide a synchronous call that returns **only the events your system decided to keep**, in the **same JSON shape** as above, sorted according to your default ranking (importance × recency).This call **must be deterministic** for a given ingestion batch so our tests can assert exact membership and ordering.

Notes

- You may enrich the schema internally, but the interface exposed to our harness must respect the shape above.
- Our tests will push several batches and then query the retrieval endpoint to confirm that false positives/negatives are handled correctly and that ranking is stable.
- If you expose more than one protocol, document which one we should target; we will use only a single interface for grading.

Engineering Practices & Design:

- Write clean, modular, readable code.
- Include basic functionality tests.
- Implement logging to monitor system health and performance.

Bonus Question:

- How would you evaluate the efficiency and correctness of your news retrieval and filtering process?

Submission Guidelines

Time Expectation

This assessment is designed to be completed in about 4-10 hours. It’s okay if you take a bit longer, but we are not expecting a perfect or complete production solution. It’s fine to leave minor to-dos or notes on what you’d improve with more time. Focus on delivering the core ideas and demonstrating your skills within a reasonable time frame.

Allowed Tools & Languages

You may use any programming language you are comfortable with (Python, Java, C#, etc. are all fine, as are scripting or query languages where appropriate). Feel free to leverage common libraries, frameworks, cloud services, and public resources as needed.

The goal is to simulate a real problem-solving scenario, so using documentation or standard tools is encouraged. (For example, using AWS managed services or Python ML libraries is acceptable.) If you make any non-obvious assumptions or use any significant external tools, note them in your reflection.

Format

You can submit your code and answers as a zip or repository link. Include:

Source code

Organize it logically (with a README if needed for any setup). Your written reflection document (in PDF or Markdown) explaining your approach. You can either combine both questions' discussion into one document with separate sections, or provide two separate write-ups. Ensure this write-up covers the reasoning behind your architecture and implementation decisions, any assumptions, and how you tested or verified your solution. If you created any architecture diagrams or drawings, please include them as well.

Use of External Resources

You are free to use online documentation, libraries, AI tools, etc. This is an open-book scenario, much like real development. However, all work submitted should be your own. If you do incorporate any snippets of code or ideas that are not yours (e.g., from a blog), please cite or briefly mention that in your reflection. We are okay with you using common utilities or templates – just be sure you understand and can explain anything in your submission, as we may discuss your solution in a follow-up interview. Direct copy-paste solutions without understanding are likely to perform poorly in later rounds.

Assumptions

You may make reasonable assumptions wherever requirements are not fully specified (in fact, part of the test is identifying what assumptions are needed). Just state these assumptions in your reflection. For instance, if you assume a certain format for input data or a certain limitation in scope, that's fine as long as you document it.

Creativity and Scope

The problem is open-ended by design. If you have an idea to extend or improve the solution, feel free to mention it in your reflection under a "Future Work" or "Possible Improvements" section.

Submission Deadline

Please submit your response via email within 7 days to the recruiter who provided you with this assignment. After we receive your submission, our team will evaluate it. Strong submissions will be invited to the next stage.

Thank you for taking the time to work on this assessment. We appreciate your interest in Nexthink and look forward to reviewing your work!